Data Structures and Algorithms Laboratory Workbook

Exercise 2: E-commerce Platform Search Function

Scenario: Optimizing search workflows across an e-commerce platform. This implementation maps a clean object layout for a `Product` dataset and comparatively benchmarks traditional **Linear Search** against an optimized **Binary Search** algorithm.

SOURCE CODE IMPLEMENTATION

```
import java.util.Arrays;
import java.util.Comparator;

// 1. Setup: Define the Product class with search attributes
class Product {
    int productId;
    String productName;
    String category;

    public Product(int productId, String productName, String category) {
        this.productId = productId;
        this.productName = productName;
        this.category = category;
    }

    @Override
    public String toString() {
        return "Product{id=" + productId + ", name='" + productName + "', category='" +
category + "'}";
    }
}

public class ECommerceSearch {

    // 2. Implementation: Linear Search Algorithm
    // Works on unsorted arrays. O(n) worst-case time complexity.
    public static Product linearSearch(Product[] products, int targetId) {
        for (Product product : products) {
            if (product.productId == targetId) {
                return product; // Found
            }
        }
        return null; // Not found
    }

    // 3. Implementation: Binary Search Algorithm
```

```
    // Requires a sorted array. O(log n) worst-case time complexity.
    public static Product binarySearch(Product[] products, int targetId) {
        int left = 0;
        int right = products.length - 1;

        while (left <= right) {
            int mid = left + (right - left) / 2; // Prevents potential integer overflow

            if (products[mid].productId == targetId) {
                return products[mid]; // Found
            }
            if (products[mid].productId < targetId) {
                left = mid + 1; // Target is in the right half
            } else {
                right = mid - 1; // Target is in the left half
            }
        }
        return null; // Not found
    }

    public static void main(String[] args) {
        // Setup raw array for linear search (unsorted context)
        Product[] products = {
            new Product(103, "Laptop", "Electronics"),
            new Product(101, "Phone", "Electronics"),
            new Product(105, "Shoes", "Apparel"),
            new Product(102, "Book", "Books")
        };

        // Test Linear Search
        System.out.println("--- Linear Search ---");
        Product result1 = linearSearch(products, 105);
        System.out.println("Result: " + result1);

        // Test Binary Search (Must explicitly sort the array first by productId)
        System.out.println("\n--- Binary Search ---");
        Arrays.sort(products, Comparator.comparingInt(p -> p.productId));

        Product result2 = binarySearch(products, 105);
        System.out.println("Result: " + result2);
    }
}
```

CODE EXPLANATION & ANALYSIS

Linear Search Logic: Iterates through the collection elements sequentially. It offers structural simplicity and holds no structural assumptions (does not require sorted keyspace boundaries), but metrics thin down heavily into a time complexity layout of $O(n)$, making execution unviable across large-scale enterprise tracking with millions of operational records.

Binary Search Logic: Highly optimized pipeline designed to repeatedly bisect the remaining sorted tracking range in halves over every successive validation loop. This locks worst-case time complexity strictly to an upper ceiling of $O(\log n)$.

Platform Fit: Binary search models are standard for active production environments. Querying across an inventory profile of $n = 1{,}000{,}000$ objects processes within roughly 20 evaluation cycles using Binary Search, compared against up to a full million sequential scans natively executed by linear frameworks.

Exercise 7: Financial Forecasting

Scenario: Engineering financial growth modeling matrices utilizing structural multi-period compounding logic. This module builds out a clean comparison layout exploring **Linear Recursion Structures** and their highly optimized **Iterative Loops** design patterns.

SOURCE CODE IMPLEMENTATION

```
public class FinancialForecasting {

    // 1. Setup & Implementation: Recursive method to calculate future value
    // Compounding formula modeled via recursion: Future Value = Present Value * (1 + growthRate)^periods
    public static double calculateFutureValue(double presentValue, double growthRate, int periods) {
        // Base Case: If no time periods are left, the target initial baseline remains fixed
        if (periods <= 0) {
            return presentValue;
        }

        // Recursive Case: Compound value parameters across one frame, reducing steps remaining
        double valueNextPeriod = presentValue * (1 + growthRate);
        return calculateFutureValue(valueNextPeriod, growthRate, periods - 1);
    }

    // 2. Optimization: An optimized iterative approach to prevent stack overflow
    public static double calculateFutureValueOptimized(double presentValue, double growthRate, int periods) {
        double futureValue = presentValue;
        for (int i = 0; i < periods; i++) {
            futureValue *= (1 + growthRate);
        }
        return futureValue;
    }

    public static void main(String[] args) {
        double initialInvestment = 1000.0; // $1,000 principal amount
        double annualGrowthRate = 0.05;    // 5% compounding performance metric
        int years = 10;                    // Total tracking projection lifespan

        // Test Recursive Solution execution flow
        double recursiveResult = calculateFutureValue(initialInvestment, annualGrowthRate, years);
        System.out.printf("Recursive Forecast Value after %d years: $%.2f%n", years, recursiveResult);

        // Test Optimized Iterative Solution framework
```

```
        double iterativeResult = calculateFutureValueOptimized(initialInvestment,
annualGrowthRate, years);
        System.out.printf("Iterative (Optimized) Value after %d years: $%.2f%n", years,
iterativeResult);
    }
}
```

CODE EXPLANATION & ANALYSIS

Recursive Mechanics: The logic leverages growth ratios across the present interval to map a rolling updated principal baseline, which feeds recursively into successive evaluation stack frames while subtracting 1 from the tracking countdown pointer over each link block. Execution breaks cleanly once the validation counter triggers the baseline check rule ($periods \leq 0$).

Time Complexity: The analytical profile tracking sequential parameters across this clean linear layout records as $O(t)$, where t mirrors the requested growth time segments directly.

Optimization Requirements: Recursion models clean multi-period compounding mathematics beautifully, but introduces structural runtime overhead from allocating persistent frame structures inside the application memory stack. For highly granular or deep historical calculations (e.g., thousands of timeline tracking data intervals), moving workflows to an iterative configuration completely lowers space requirements from $O(t)$ stack allocation down to a static $O(1)$ configuration. This eliminates structural risks for runtime `StackOverflowError` crashes while streamlining resource execution profiles.